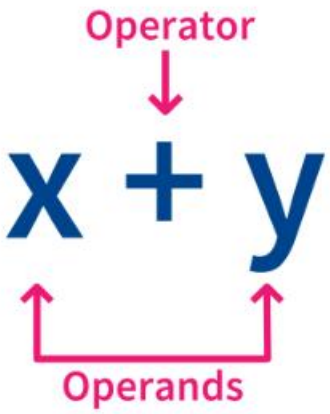
🛠 What is an Operator?

An **operator** is a special symbol that directs the computer to perform a specific mathematical, relational, or logical manipulation on one or more values.

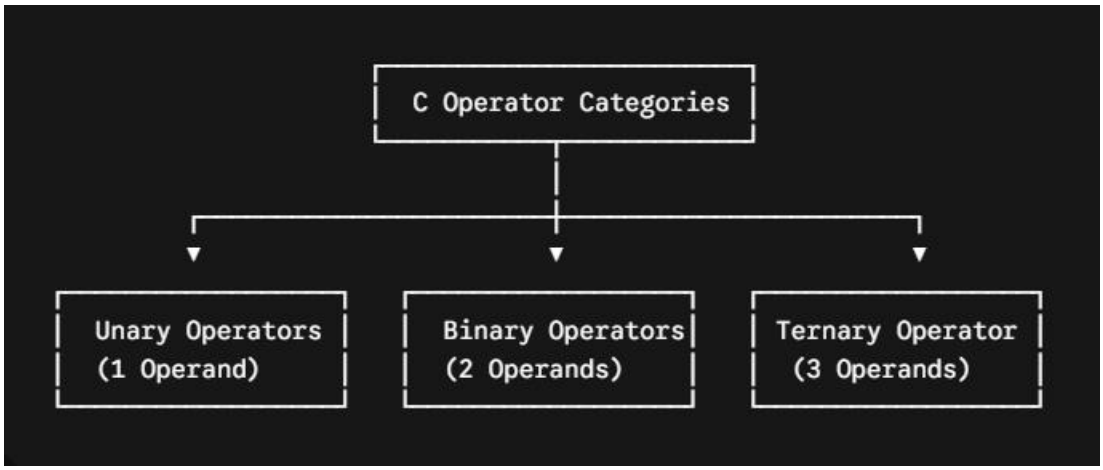
> **Operand:** The data or variable/constant upon which the operation is executed.
>
> **Operator:** The symbol executing the operation.



In the expression a + b, + is the **operator**, while a and b are the **operands**.

🗺 Classification of Operators (By Operand Count)

Operators are primarily classified based on the number of operands they require to function:



1. Unary Operators (Requires 1 Operand)

A **Unary Operator** acts upon a single operand to produce a new value.

Core Unary Operators Matrix :

Operator	Name	Meaning & Behavior	C Code Example
++	Increment	Increases integer value by 1.	int a = 10; a++; ----> a becomes 11
--	Decrement	Decreases integer value by 1.	int a = 10; a--; ----> a becomes 9
-	Unary Minus	Negates the sign of the value.	int a = 10; int b = -a; ----> b is -10
!	Logical NOT	Inverts a boolean/truth evaluation.	!(5 > 2) ----> !(1) becomes 0 (False)
&	Address-of	Returns the physical memory location pointer.	&a; ----> Returns address code (e.g., 0x7ffe)
*	Indirection	De-references a pointer to access its value.	Used in pointer value access.
sizeof	Size Measurement	Outputs the variable or type size in bytes.	sizeof(int); ----> Evaluates to 4 bytes

2. Binary Operators (Requires 2 Operands)

A **Binary Operator** operates on two distinct operands separated by the operator symbol.

A. Arithmetic Operators

Used to execute standard mathematical computations.

Operator	Operation	Mathematical Example	Output / Result

Operator	Operation	Mathematical Example	Output / Result
+	Addition	10 + 20	30
-	Subtraction	20 - 5	15
*	Multiplication	10 * 5	50
/	Division	20 / 4	5 (Quotient result)
%	Modulus	10 % 3	1 (Remainder result)

B. Relational & Logical Operators

Relational operators evaluate comparisons , whereas logical operators combine multiple conditional properties.

Relational (>, <, >=, <=, ==, !=) : Compares two values. For example, 10 > 5 evaluates to 1 (True).

Logical AND (&&) : True *only* if both conditions evaluate to true. Example: (a > 5) && (b < 10).

Logical OR (||) : True if *at least one* condition evaluates to true. Example: (a > 5) || (b < 10).

C. Assignment Operators

Used to write or update values inside variables.

Simple Assignment: a = 5;

Compound Shorthands (+=, -=, *=, /=, %=): Shorthand notation.

a += 5; is functionally identical to **a = a + 5;**

D. Bitwise Operators (Binary Sub-Category)

Bitwise operators manipulate the raw binary digits (**0s and 1s**) of an integer directly.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	XOR
~	Complement
<<	Left Shift
>>	Right Shift

Bitwise Operator Understanding :

Bit A	Bit B	A & B (AND)	A | B (OR)	A ^ B (XOR)	~ A (NOT)
0	**0**	0	0	0	1
0	**1**	0	1	1	1
1	**0**	0	1	1	0
1	**1**	1	1	0	0

Key Rule :

Bitwise AND (&) : Result is 1 only when both bits are 1.

Bitwise OR (|) : Result is 1 if at least one bit is 1.

Bitwise XOR (^) : Result is 1 when bits are different.

Bitwise NOT (~) : Makes every bit opposite.

🗄️ Understanding Bitwise right shift :

1. Decimal to binay Conversion { Shortcut }

Convert decimals to their active binary representations using power-of-two markers:

Power Weight:	128	64	32	16	8	4	2	1
Decimal 47:	0	0	1	0	1	1	1	1

Int x = 47 ;

Binary Representation of 47 (represented in a standard 32-bit integer grid):

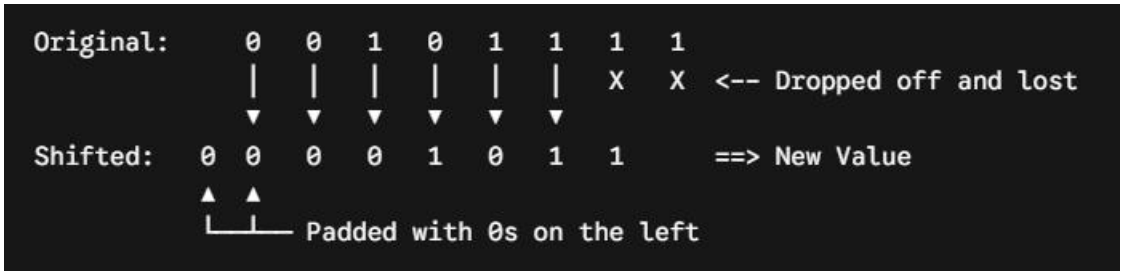
00000000000000000000000000101111 ---> As x is an int type so it takes 4 bytes or 32 bit space

Bitwise Right Shift (>>): x = 47 >> 2;

Right shifting moves all bits to the right by the specified position count, dropping trailing values off the edge.

Execution of Right Shift by 2 (>> 2):

The entire sequence shifts rightward twice. The last two bits (11) drop off, and two zero-padded bits roll into the leading left spaces.



00000000000000000000000000001011 ----> 11 new value

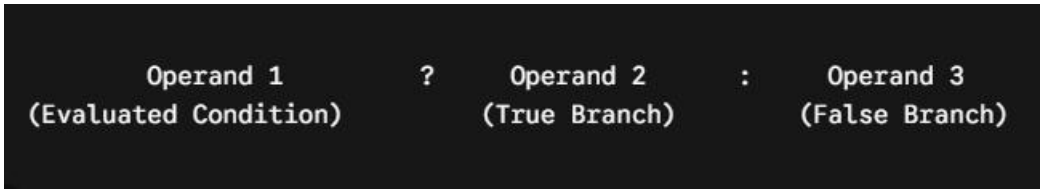
⚠ **Critical Architectural Rule:** > When processing shifts with **positive numbers**, empty slots are consistently padded with 0. When operating with **negative numbers**, padding behavior depends on your computer's compiler setup.

4. Ternary Operator (Requires 3 Operands)

C contains exactly one operator that manages three distinct operands: the **Conditional/Ternary Operator** (? :). It operates as an inline, shorthand alternative to standard if-else blocks.

Syntax Structure :

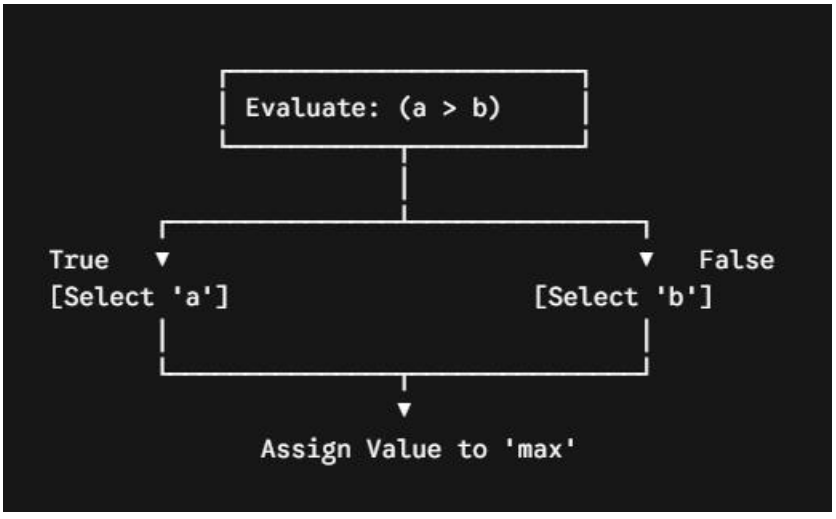
(condition) ? expression1 : expression2;



◇ Logic Flow Diagram

int a =10 , b = 20 ;

(a>b) ? a : b ;

- 

📋 Complete C Operator Cheat Sheet

Functional Group	Operators	Functional Role
Arithmetic	+, -, *, /, %	Performs basic math calculations.
Relational	>, <, >=, <=, ==, !=	Runs value-to-value comparisons.
Logical	&&, ||, !	Evaluates combined boolean conditions.
Assignment	=, +=, -=, *=, /=, %=	Writes / alters tracking data inside variables.
Inc / Dec	++, --	Shorthand adjustment loops to add or subtract 1.
Conditional	? :	Evaluates inline boolean choices.
Bitwise	&, |, ^, ~, <<, >>	Performs precise manipulation at bit-level scale.
Special	sizeof(), &, *	Measures memory size and handles hardware pointers.